

resources that were used by the process after the `exit()` system call.

- (6) **Open** : It is the system call to open a file. This system call just opens the file, to perform operations such as read and write, we need to execute different system call to perform the operations.
- (7) **Close** : This system call closes the opened file.
- (8) **Stat** : `stat` serves as a versatile utility that fetches and displays data about files and directories. From determining file types to monitoring file system usage, `stat` helps users gain insights into various file attributes which are crucial for system management and file handling operations.
- (9) **Uname** : Let's start with the basics. The term "uname" stands for "Unix Name," and the command itself is designed to provide you with key details about your Linux system. It's like asking your computer, "Hey, who are you, and what are you made of?" The answers you get can help you understand your system's kernel version, operating system, hardware architecture, and more. This command `'uname'` displays the information about the system.

Options and Examples of `'uname'` Command in Linux

Options	Description
<code>'-a' or '--all'</code>	Displays all available information.
<code>'-s' or '--kernel-name'</code>	Shows the kernel name.
<code>'-n' or '--nodename'</code>	Displays the network (domain) name of the machine.
<code>'-r' or '--kernel-release'</code>	Shows the kernel release.
<code>'-v' or '--kernel-version'</code>	Displays the kernel version.
<code>'-m' or '--machine'</code>	Shows the machine hardware name.
<code>'-p' or '--processor'</code>	Displays the processor type or unknown."
<code>'-i' or '--hardware- platform'</code>	Shows the hardware platform or unknown."
<code>'-o' or '--operating-system'</code>	Displays the operating system.

(VII) Recourses Required

- Computer System with basic configuration. Linux/Ubuntu/ CentOS /any other open source Operating System.
- Software: Turbo C/ vi editor/ Any open source Python IDE such IDLE, Jupyter Notebook, Spyder, PyCharm etc.)

(VIII) Precautions to be followed

- (1) Handle computer system with care.
- (2) Strictly follow the instructions for writing, compiling, and executing the program.
- (3) Don't forget to save file before execution.
- (4) Follow safety Practices

(IX) Conclusion

Successfully executed System Call commands in Linux such as `fork()`, `exec()`, `getpid`, `pipe`, `exit`, `open`, `close`, `stat`, `uname`

(X) Practical Related Questions

Note: Below given are few sample questions for reference. Teachers must design more such questions to ensure the achievement of identified CO.

- (1) Write use of system calls.
- (2) Write system calls related to Process management.
- (3) Write a command which returns a PID.
- (4) What are various options for `uname`.
- (5) Explain about `exit` command.
- (6) List any four system calls and write their use

[Space to Write for Answers]

- 1) 1. Process management : Create, execute and terminate processes.
2. File management : Create, open, read, write and close files.
3. Device management : Access and control hardware devices such as printers and disks.
4. Memory management : Allocate and release memory for programs.
5. Communication : Enable communication between processes and between computers.
6. Security & Protection : Control access to system resources and protect data.

- 2) 1. Create Process
2. Terminate Process
3. Load and execute
4. Wait
5. Get process attributes
6. Set process attributes
7. Allocate and Free Memory

3) echo \$\$

- 4) uname -a : Displays all available system information.
- uname -s : Displays the kernel name
- uname -n : Displays the network/host name
- uname -r : Displays the kernel release
- uname -v : Displays the kernel version
- uname -m : Displays the machine hardware name
- uname -p : Displays the processor type
- uname -i : Displays the hardware platform
- uname -o : Displays the operating system.

5) The `exit` command is used to terminate the current shell session or exit from a terminal.

Syntax: `exit`

It can also be used with a status code: `exit 0`

`exit 0` indicates successful termination

A non-zero value such as `exit 1` generally indicates an error or unsuccessful termination

Example: `$ exit`

This closes the current terminal shell or logs the user out of the shell session.

6) 1. `fork()` : Creates a new process

2. `exec()` : Executes a new program/process

3. `wait()` : Makes a process wait until another process finishes.

4. `exit()` : Terminates the current process.